

System Design Fundamentals — Intermediate Track

Target: Engineers with 2-5 years of experience who understand the basics but haven't designed large-scale distributed systems.

Goal: Move from "knows the concepts" to "can design, justify, and critique real production architectures."

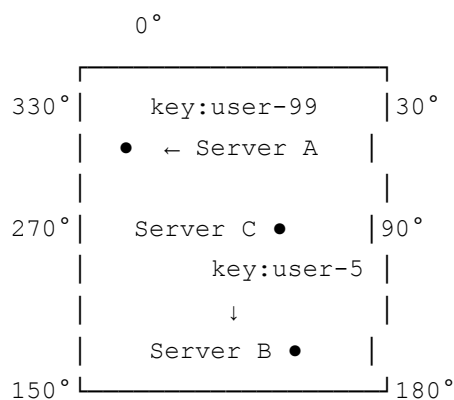
FDE Signal: This is where most FDE interviews live. You'll design systems with real trade-offs, not just define terms.

PART 1 — Scaling & Architecture Deep Dive

1. Consistent Hashing

Problem: You have N cache servers. You hash `user_id % N` to find which server holds the data. You add a server $\rightarrow N$ changes $\rightarrow$ almost all keys remap $\rightarrow$ cache miss storm.

Solution: Consistent Hashing maps both servers and keys to positions on a ring. Adding/removing a server only affects keys between that server and its predecessor.



Each key maps clockwise to the nearest server.

Add Server D between A and B $\rightarrow$ only A-to-D range remaps.

Virtual Nodes (vnodes): Each server is assigned multiple positions on the ring. Prevents uneven distribution when servers have different capacities.

Real Example — Amazon DynamoDB & Apache Cassandra Both use consistent hashing with vnodes. In Cassandra, each node owns multiple token ranges. When you add a node, Cassandra rebalances only the affected ranges, not the entire dataset.

Real Example — CDN Routing Akamai uses a variant of consistent hashing to route requests to edge servers. Your request for `video.cdn.com` consistently maps to the same edge PoP (for cache efficiency) unless that PoP is down.

Interview: Design a distributed cache using consistent hashing

Architecture:

Client → [Consistent Hash Router] → [Cache Node 1..N]

↑

Knows ring topology via:

- ZooKeeper / etcd (service discovery)
- Client-side ring (client tracks ring)

On node failure:

- Successor node takes over the failed node's range
- Read requests → cache miss → fetch from DB → repopulate

On node addition:

- Only preceding range remaps
- Previous node streams its data to new node (rebalance)

Anti-patterns:

- Too few vnodes → uneven load distribution
- Not handling node failure in router → requests to dead node hang

2. Sharding (Data Partitioning)

Sharding splits a large dataset across multiple database instances (shards). Each shard owns a subset of data.

Sharding Strategies:

Range-based Sharding

```
Shard 1: user_id 1-1,000,000
Shard 2: user_id 1,000,001-2,000,000
Shard 3: user_id 2,000,001-3,000,000
```

Pro: Simple. Range queries easy.

Con: Hotspots (if new users cluster at top of range, Shard 3 is hot).

Hash-based Sharding

```
Shard = hash(user_id) % num_shards  
user_id=123 → hash=456789 → 456789 % 3 = 0 → Shard 1
```

Pro: Even distribution.

Con: Range queries require all shards. Resharding is painful.

Directory-based Sharding

```
Lookup table: user_id → shard_id  
user 123 → shard 2  
user 456 → shard 1
```

Pro: Flexible. Can reassign shards.

Con: Lookup table is a bottleneck and SPOF.

Geographic Sharding

```
US users → US shard (Virginia)  
EU users → EU shard (Frankfurt) [GDPR compliance!]  
APAC users → APAC shard (Singapore)
```

The Problem with Cross-Shard Operations

```
-- This query is easy on one DB, painful on 3 shards:  
SELECT COUNT(*) FROM orders WHERE product_id = 99;  
-- Must query all shards and sum results (scatter-gather)
```

Real Example — Instagram Sharding Instagram (before migration to Citus/PostgreSQL) sharded their user data across thousands of PostgreSQL instances using logical shards mapped to physical servers. Their shard key was `user_id`. They could query one user's data on one shard but cross-user analytics required scatter-gather.

Real Example — Uber Trips Uber shards trip data by city. All trips in New York go to the New York shard. Geo queries stay on one shard. Cross-city analytics run on data warehouse (Hive/Presto), not production shards.

Interview: Design a URL Shortener (Bit.ly)

Problem: 100M new URLs/day, 10B redirects/day

Sharding approach:

- Short code → hash → shard_id
- short code "abc123" → hash → shard 7 → lookup long URL

Schema per shard:

```
CREATE TABLE urls (  
  short_code VARCHAR(8) PRIMARY KEY,  
  long_url TEXT,  
  created_at TIMESTAMP,  
  click_count BIGINT  
);
```

Scaling:

- Reads: Cache top short codes in Redis (80% of traffic to 20% of URLs)
- Writes: Accept on any shard
- Analytics: Async stream to Kafka → ClickHouse for click count aggregation

3. Database Replication Deep Dive

Leader-Follower (Primary-Replica) Replication

```
All Writes → [Leader/Primary]  
           ↓   ↓   ↓  
           [R1] [R2] [R3] ← Read from any replica
```

WAL (Write-Ahead Log): The leader writes changes to a WAL (log on disk) before applying them. Replicas receive and replay the WAL. This ensures durability — if leader crashes mid-write, WAL allows recovery.

WAL Entry Example:

```
LSN 1001: INSERT INTO orders (id,amount) VALUES (456, 100)  
LSN 1002: UPDATE accounts SET balance = balance - 100 WHERE id = 123  
LSN 1003: COMMIT
```

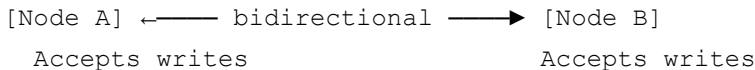
Replica applies LSN 1001, 1002, 1003 in order.

Replication Lag: The delay between leader write and replica applying it. During heavy load, replicas can fall behind by seconds or minutes. Reading from a lagging replica → stale data.

Solutions for Replication Lag:

- **Read from primary** for critical reads (e.g., “show me my balance after transfer”)
- **Read-your-writes consistency:** Route user’s reads to primary for N seconds after a write
- **Monotonic reads:** Always route same user to same replica (session affinity)

Leader-Leader (Multi-Master) Replication



Pro: Both nodes accept writes. No single point of failure for writes.

Con: **Write conflicts.** Both nodes accept conflicting writes simultaneously. Conflict resolution is hard (last-write-wins, application logic, CRDTs).

Real Example: CouchDB, DynamoDB with Global Tables, Cassandra. Used when you need writes from multiple geographic regions.

Real Example — GitHub’s MySQL Setup GitHub runs MySQL in a leader-follower configuration. For reads, they use ProxySQL to route queries. During a leader failover, they use Orchestrator to automatically promote a replica to leader and update all client connections.

4. Partitioning vs Sharding

These terms are often used interchangeably but have a distinction:

- **Partitioning:** Dividing data *within* a single database instance into logical groups (like PostgreSQL table partitions)
- **Sharding:** Distributing data across *multiple* database instances

```
-- PostgreSQL partitioning (same DB, multiple tables)
CREATE TABLE orders PARTITION BY RANGE (created_at);
CREATE TABLE orders_2024 PARTITION OF orders
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
CREATE TABLE orders_2025 PARTITION OF orders
    FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

Benefits of table partitioning:

- Drop old data fast (DROP PARTITION vs DELETE millions of rows)
 - Query planner only scans relevant partition (partition pruning)
 - Separate storage (cold data on cheap disks)
-

5. Service Discovery

In a microservices system, services need to find each other. IPs change as containers restart.

Without service discovery:

PaymentService hardcodes: `http://user-service:8080` → breaks when IP changes

With service discovery:

PaymentService asks: "Where is user-service?"

Registry says: "10.0.1.42:8080"

Client-Side Discovery

Client → [Service Registry] → Gets list of User Service instances

Client → picks instance → calls directly

Pro: Client controls load balancing. No extra hop.

Con: Client needs discovery library. Language-specific.

Server-Side Discovery

Client → [Load Balancer/API Gateway]

↓

[Service Registry]

↓

[Service Instance]

Pro: Client is simple, language-agnostic.

Con: Extra network hop.

Tools: Consul, Kubernetes (built-in DNS), Eureka (Netflix), etcd

Kubernetes DNS Service Discovery:

```
# Service named "user-service" in namespace "default"
```

```
# Accessible at: user-service.default.svc.cluster.local:8080
# Or simply: user-service:8080 within same namespace
```

6. Leader Election

When you have multiple instances of a service but need exactly one “leader” (e.g., scheduler, cron job, lock holder), you need leader election.

Why it’s hard: Network partitions can make all nodes think they’re leader (split-brain).

Algorithms:

Bully Algorithm: Highest-priority node wins. Simple but not partition-tolerant.

Raft Consensus: Leader election via quorum (majority vote). Used by etcd, CockroachDB.

1. Node starts election → becomes Candidate → requests votes
2. If majority vote → becomes Leader
3. Leader sends heartbeats to prevent new elections
4. If heartbeat missed → followers start new election

ZooKeeper (Ephemeral Nodes):

```
All nodes try to create /leader (ephemeral znode)
Only one succeeds → that node is leader
If leader dies → ZooKeeper deletes ephemeral node → others race again
```

Real Example — Kafka Controller Kafka uses ZooKeeper (being replaced by KRaft) for broker leader election. One broker is the “Controller” responsible for partition leadership. On failure, ZooKeeper detects the crash and remaining brokers elect a new controller.

Real Example — Elasticsearch Master Node Elasticsearch uses its own Raft-like protocol (Zen Discovery) for master election. The master manages cluster state, index routing tables, and shard allocation.

7. Distributed Transactions

The Problem: How do you maintain ACID across multiple services/databases?

Transfer \$100 from Bank A to Bank B:

Step 1: Deduct \$100 from Bank A's DB

Step 2: Add \$100 to Bank B's DB

What if Step 2 fails? Bank A already deducted!

2-Phase Commit (2PC)

Coordinator asks all participants: "Can you commit?"

All say Yes → Coordinator says: "Commit!"

Any says No → Coordinator says: "Rollback!"

Pro: Atomic.

Con: Coordinator is SPOF. Blocking protocol. Can leave DB "in doubt" if coordinator crashes.

SAGA Pattern — Preferred in Microservices

Instead of a global transaction, use a sequence of local transactions with compensating transactions on failure.

Choreography-based SAGA:

OrderService creates order (Event: OrderCreated)

↓

InventoryService reserves item (Event: ItemReserved OR InventoryFailed)

↓

PaymentService charges card (Event: PaymentCharged OR PaymentFailed)

↓

ShippingService creates shipment

On PaymentFailed:

InventoryService releases reservation ← compensating transaction

OrderService cancels order

Orchestration-based SAGA:

OrderSaga (orchestrator) → calls InventoryService

→ calls PaymentService

→ on failure, calls compensating transactions

Real Example — Uber SAGA Uber's payment flow uses a SAGA: request ride → reserve driver → charge payment → on failure, cancel reservation. The CADENCE workflow

engine (now Temporal) orchestrates these sagas.

8. Microservices Communication Patterns

Synchronous (Request-Response)

Service A — HTTP/gRPC — Service B — Response — Service A

Use for: Needs immediate response. Simple flows.

Risk: Service B slow → Service A slow → cascading latency.

Asynchronous (Event-Driven)

Service A — publish event — [Message Bus] — Service B processes
Service C processes

Use for: Non-blocking. Fan-out to multiple consumers. Decoupled.

Risk: Eventual consistency. Harder to debug. Error handling complexity.

Fan-Out / Fan-In

Fan-Out: One request → multiple parallel service calls

Fan-In: Wait for all to complete → aggregate results

Example: Product page load:

```
→ [User Service] _____
→ [Inventory Service] _____ } Aggregate
→ [Review Service] _____ } → Response
→ [Recommendation Service] _____
```

Real Example — Amazon Product Page Amazon's product detail page makes 150+ service calls, fanned out in parallel. The page renders as results return (partial rendering). Services have SLAs — if Recommendation Service takes >100ms, render page without recommendations.

9. API Design Patterns

Versioning Strategies

URI Versioning: /api/v1/users, /api/v2/users (most common, visible)
Header Versioning: Accept: application/vnd.api+json; version=2 (cleaner)
Query Param: /api/users?version=2 (simple, cache-unfriendly)

REST Best Practices

Use nouns, not verbs:

- ✓ GET /orders/123
- ✗ GET /getOrder?id=123

Use HTTP methods correctly:

GET = read (safe, idempotent)
POST = create (not idempotent)
PUT = full replace (idempotent)
PATCH = partial update (idempotent)
DELETE = remove (idempotent)

Return consistent error format:

```
{
  "error": "order_not_found",
  "message": "Order 123 does not exist",
  "request_id": "abc-123"
}
```

Pagination Patterns

Offset Pagination (simple, but slow for large offsets):

GET /users?page=1000&limit=20
→ SELECT * FROM users LIMIT 20 OFFSET 20000 ← full table scan to 20000!

Cursor Pagination (recommended for large datasets):

GET /users?after=eyJlc2VyX2lkIjoxMDAwfQ==&limit=20
→ SELECT * FROM users WHERE id > 1000 LIMIT 20 ← uses index!
(cursor is base64 of last seen id/timestamp)

Keyset Pagination (same as cursor, using last seen value):

GET /posts?created_before=2024-01-15T10:30:00Z&limit=20

10. Message Queue Deep Dive

Kafka Architecture

Producers → [Topic: orders] → Consumers

Partition 0: [msg1] [msg3] [msg5] ← Consumer Group A

Partition 1: [msg2] [msg4] [msg6] ← Consumer Group A

Partition 2: [msg7] [msg8] [msg9] ← Consumer Group B

Key concepts:

- **Topic:** Category/stream name
- **Partition:** Unit of parallelism. More partitions = more consumers reading in parallel
- **Consumer Group:** Multiple consumers sharing partition assignment
- **Offset:** Position of each consumer in each partition
- **Retention:** Messages kept for N days regardless of consumption

Delivery guarantees:

Guarantee	Meaning	Use case
At-most-once	May lose messages, no duplicates	Metrics
At-least-once	No loss, may duplicate	Default. Use idempotent consumers
Exactly-once	No loss, no duplicates	Payments (Kafka transactions)

Dead Letter Queue (DLQ)

Normal flow: Queue → Consumer → Process ✓

On failure:

Queue → Consumer → Process fails → retry 3x → DLQ (Dead Letter Queue)

DLQ:

- Stores messages that failed after N retries
- Engineers inspect DLQ for bugs
- Can replay messages after fixing bug
- Alert on DLQ depth metric

Real Example — Stripe Webhook Delivery Stripe sends webhooks (payment events)

via a queue. On consumer failure, they retry with exponential backoff: 1m, 5m, 30m, 2h, 8h. After all retries fail, the event goes to a "failed" state. You can manually replay from the dashboard.

11. Service Mesh

A service mesh manages service-to-service communication at the infrastructure level (not application code).

Without service mesh:

ServiceA — (code handles retry, mTLS, trace) — ServiceB

With service mesh (Istio/Linkerd):

```
ServiceA — [Envoy Sidecar] — [Envoy Sidecar] — ServiceB
```

handles: mTLS, retry, timeout, circuit break, traces
(zero application code changes)

What a service mesh provides:

- **mTLS:** Encrypted, authenticated service-to-service communication
- **Traffic management:** Canary deployments, A/B testing, traffic shifting
- **Observability:** Automatic tracing without code changes
- **Resilience:** Retries, timeouts, circuit breakers at infrastructure level
- **Policy:** Rate limiting, access control between services

Control Plane vs Data Plane:

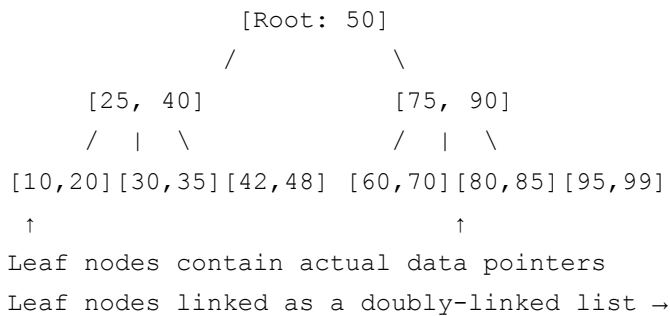
- **Data Plane:** Sidecar proxies (Envoy) next to each service. Handle actual traffic.
- **Control Plane:** Istiod, manages sidecar configuration. Pushes policies to sidecars.

PART 2 – Database Internals

12. B+ Tree Index Internals

B+ Trees are the default index structure in PostgreSQL, MySQL InnoDB, and most relational databases.

Structure:



Properties:

- All data is in leaf nodes
- Leaf nodes linked → efficient range scans
- Balanced: all leaves at same depth
- Height is $O(\log n)$ — typically 3-4 levels for millions of rows
- Each node is a disk page (typically 8KB or 16KB)

Why range queries are fast:

```
SELECT * FROM orders WHERE created_at BETWEEN '2024-01-01' AND '2024-01-31';
-- B+Tree finds first leaf with date >= 2024-01-01
-- Then walks linked list until date > 2024-01-31
-- No tree traversal after finding start!
```

13. Transaction Isolation Levels

Four isolation levels defined by SQL standard, from weakest to strongest:

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Use case
Read Uncommitted	✓ possible	✓ possible	✓ possible	Analytics (rare)
Read Committed	✗ prevented	✓ possible	✓ possible	PostgreSQL default
Repeatable Read	✗ prevented	✗ prevented	✓ possible	MySQL InnoDB default

Serializable	× prevented	× prevented	× prevented	Banking
--------------	-------------	-------------	-------------	---------

Dirty Read: Read another transaction's uncommitted data

Non-Repeatable Read: Read same row twice, get different values (another transaction committed between reads)

Phantom Read: Re-run a query, get different rows (another transaction inserted/deleted between reads)

Real Example:

```
-- Read Committed (PostgreSQL default):
BEGIN;
SELECT balance FROM accounts WHERE id = 1; → $100
-- Other transaction commits: balance → $0
SELECT balance FROM accounts WHERE id = 1; → $0 (non-repeatable read!)
COMMIT;

-- Repeatable Read:
BEGIN;
SELECT balance FROM accounts WHERE id = 1; → $100
-- Other transaction commits: balance → $0
SELECT balance FROM accounts WHERE id = 1; → still $100 (repeatable!)
COMMIT;
```

14. OLAP vs OLTP

	OLTP	OLAP
Purpose	Operational (run the business)	Analytical (understand the business)
Queries	Simple, small (by PK or index)	Complex, large (aggregations over millions of rows)
Transactions	Short (ms)	Long (seconds to minutes)
Optimization	Row-oriented storage	Columnar storage
Examples	PostgreSQL, MySQL, DynamoDB	BigQuery, Redshift, Snowflake, ClickHouse
Latency	<10ms	Seconds to minutes

Data	Current operational data	Historical data warehouse
------	--------------------------	---------------------------

Columnar Storage (OLAP):

Row storage (OLTP):

Row 1: [id=1, name="Alice", age=30, salary=100k]

Row 2: [id=2, name="Bob", age=25, salary=80k]

Columnar storage (OLAP):

id column: [1, 2, 3, 4...]

name column: ["Alice", "Bob", ...]

salary column: [100k, 80k, 120k...] ← compressed, fast to scan

`SELECT AVG(salary) FROM employees` **only scans the salary column in columnar. In row storage, it must read entire rows.**

Real Example — Netflix Analytics Netflix runs OLTP on PostgreSQL/Cassandra (serve the app). Analytics run on Apache Iceberg + Spark on S3 (compute recommendations, analyze play patterns). They separate the two completely — no analytics queries ever hit production databases.

15. Bloom Filters

A Bloom filter is a probabilistic data structure that answers: "Is this item **definitely not** in the set, or **possibly** in the set?"

False positives: possible (says "yes" when answer is "no")

False negatives: impossible (never says "no" when answer is "yes")

How it works:

Bit array of size M, initialized to all 0s.

To add "user-123":

hash1("user-123") = 5 → set bit 5 = 1

hash2("user-123") = 12 → set bit 12 = 1

hash3("user-123") = 7 → set bit 7 = 1

To check "user-456":

hash1("user-456") = 5 → bit 5 = 1 ✓

hash2("user-456") = 3 → bit 3 = 0 ✗

→ "user-456" DEFINITELY NOT in set

To check "user-789":

hash1("user-789") = 5 → bit 5 = 1 ✓

hash2("user-789") = 12 → bit 12 = 1 ✓

hash3("user-789") = 7 → bit 7 = 1 ✓

→ POSSIBLY in set (might be false positive)

Real Uses:

- **Cassandra:** Before reading from disk, check Bloom filter. If “definitely not”, skip disk I/O.
- **Chrome:** Checks if URL is in list of malicious sites using Bloom filter before hitting server.
- **CDN cache:** “Is this URL definitely not in cache?” → skip cache check entirely.
- **Database join optimization:** Filter out rows that definitely won't join.

16. Redis Deep Dive

Redis is an in-memory data structure server. Not just a cache — it's a data platform.

Data Structures:

```
String: SET user:123:name "Alice"  GET user:123:name
Hash:   HSET user:123 name Alice age 30  HGETALL user:123
List:   LPUSH queue task1  RPOP queue (queue)
Set:    SADD tags:post:1 redis caching  SMEMBERS tags:post:1
Sorted Set: ZADD leaderboard 1500 alice  ZRANGE leaderboard 0 10 WITHSCORES
Bitmap: SETBIT active_users 123 1  BITCOUNT active_users
HyperLogLog: PFADD daily_visitors user1 user2  PFCOUNT daily_visitors
Streams: XADD events * type click url /home  XREAD COUNT 10 STREAMS events 0
```

Use cases:

Use case	Redis structure	Example
Session store	String/Hash	User session: HSET session:abc user_id 123
Rate limiting	String + INCR + EXPIRE	Count requests per user

Leaderboard	Sorted Set	Top players by score
Pub/Sub	Pub/Sub	Real-time notifications
Queue	List	LPUSH/BRPOP job queue
Distributed Lock	String + SET NX	Only one worker runs job
Cache	String/Hash	Cache DB query results
Counting	String + INCR	Page view counters

Distributed Lock with Redis (Redlock):

```
# Acquire lock (SET NX = Set if Not eXists)
success = redis.set("lock:job-123", "worker-1", nx=True, ex=30)
# nx=True: only set if key doesn't exist
# ex=30: auto-expire in 30 seconds (release if worker crashes)

if success:
    try:
        process_job()
    finally:
        # Release lock only if we own it
        lua_script = """
        if redis.call("get", KEYS[1]) == ARGV[1] then
            return redis.call("del", KEYS[1])
        end
        """
        redis.eval(lua_script, 1, "lock:job-123", "worker-1")
```

Anti-patterns:

- **Using Redis as primary database:** Redis is RAM. Data can be lost on restart unless RDB/AOF enabled.
- **Storing large values:** Redis is designed for small key-value pairs, not 10MB blobs.
- **Not setting TTL:** Memory fills up. Every cached value needs a TTL.

17. Hot/Cold Storage

Data has different access patterns over time. Tiered storage matches cost to access

frequency.

Hot Storage (Frequently accessed):

- NVMe SSD, in-memory (Redis)
- \$\$\$: High cost, high performance
- Example: Current user sessions, today's orders

Warm Storage (Occasionally accessed):

- SSD-backed databases, S3 Standard
- \$\$: Medium cost, medium performance
- Example: Last 90 days of orders

Cold Storage (Rarely accessed):

- HDD, S3 Glacier, tape
- \$: Low cost, high latency (minutes to retrieve)
- Example: 7-year archive for compliance

Real Example — Netflix Video Storage:

Hot: Popular new releases → CDN edge nodes (SSD)

Warm: Content released 1-3 years ago → CDN origin (S3 Standard)

Cold: Old catalog, master files → S3 Glacier

Lifecycle Policies (S3):

Day 0: Object created → S3 Standard (hot)

Day 30: Transition → S3 Standard-IA (warm, infrequent access)

Day 90: Transition → S3 Glacier Instant Retrieval

Day 365: Transition → S3 Glacier Deep Archive (\$0.00099/GB/month!)

Day 2555 (7 years): Delete (compliance period over)

Data Retention: Many industries have legal requirements:

- Healthcare (HIPAA): 6 years
- Financial (SOX): 7 years
- EU (GDPR): "as long as necessary" — must also support right to erasure

PART 3 — Reliability & Observability

18. Distributed Tracing

In a microservices system, a single user request touches dozens of services. Distributed tracing tracks the journey.

```
Request: GET /checkout
```

```
Trace ID: abc-123
```

```
└─ Span: API Gateway (10ms)
  │   └─ Span: Auth Service (5ms)
  │   └─ Span: Cart Service (20ms)
  │       └─ Span: DB Query (15ms)
  │       └─ Span: Redis Cache (2ms)
  │       └─ Span: Payment Service (150ms) ← bottleneck!
  │           └─ Span: Stripe API Call (140ms)
└─ Total: 185ms
```

Implementation (OpenTelemetry):

```
from opentelemetry import trace
tracer = trace.get_tracer("checkout-service")

def process_checkout(cart_id):
    with tracer.start_as_current_span("process-checkout") as span:
        span.set_attribute("cart.id", cart_id)
        span.set_attribute("user.id", current_user.id

        with tracer.start_as_current_span("validate-payment"):
            result = payment_service.validate(cart)

        with tracer.start_as_current_span("update-inventory"):
            inventory.reserve(cart)
```

Tools: Jaeger, Zipkin, AWS X-Ray, Datadog APM, Honeycomb, Grafana Tempo

FDE Note: Anthropic's Claude API serves many concurrent requests. Each request generates a trace through: rate limiter → queue → inference cluster → safety filter → response. Tracing is how you find why P99 latency is 10x higher than P50.

19. Chaos Engineering

Deliberately inject failures in production to discover weaknesses before real incidents do.

Netflix Chaos Monkey → Simian Army:

Chaos Monkey: Randomly kills EC2 instances
Chaos Gorilla: Kills entire AWS availability zone
Chaos Kong: Kills entire AWS region
Latency Monkey: Adds artificial latency between services
Conformity Monkey: Shuts down non-conforming instances

Principles:

1. **Hypothesis:** "Our system degrades gracefully when DB replica fails"
2. **Blast radius control:** Start in staging, then production off-hours, with kill switch
3. **Steady state definition:** Define what "normal" looks like (error rate, latency)
4. **Inject failure:** Kill replica, add latency, corrupt network packets
5. **Observe:** Does system match hypothesis?
6. **Learn and fix:** Remediate discovered weaknesses

GameDay: Planned chaos exercise with full team, simulating real incident. Fire drills for engineers.

20. SLO Error Budgets in Practice

SLO: 99.9% availability per month

Monthly minutes: 43,800

Allowed downtime: $43,800 \times 0.001 = 43.8$ minutes/month

Error Budget Tracking:

Week 1: 0 downtime → Budget remaining: 43.8 min

Week 2: Deploy bug → 5 min outage → Budget: 38.8 min

Week 3: DB failover → 12 min → Budget: 26.8 min

Week 4: Budget low → Freeze deployments, focus on reliability

Reset monthly.

Error Budget Policy:

- Budget healthy (>50%): Deploy freely, experiment
- Budget <25%: Review all deployments. No risky changes.
- Budget exhausted: Feature freeze. Only reliability work.

Real Example — Google SRE Google's SRE book defined this system. Each SRE team has error budget. If Gmail goes down for 10 minutes, that's a chunk of annual budget. SREs and developers negotiate: if developers want to ship fast, they must maintain their error budget.

21. Canary Deployments & Blue-Green

Blue-Green Deployment

```
Blue (Current): [v1.0] ← 100% traffic
Green (New):    [v1.1] ← 0% traffic
```

```
Deploy to Green, test. Then:  
Blue (Old): [v1.0] ← 0% traffic  
Green (New): [v1.1] ← 100% traffic
```

Rollback: instantly switch back to Blue.
Cost: 2x infrastructure during transition.

Canary Deployment

```
v1.0: ██████████ 90% traffic (stable)
v1.1: █          10% traffic (canary)
```

```
Monitor canary error rate and latency.
If good → gradually increase to 25%, 50%, 100%.
If bad → route 0% to canary, investigate.
```

Progressive Delivery with Feature Flags:

```
if feature_flag("new-checkout-flow", user_id):
    # 10% of users see new flow
    return new_checkout()
else:
    return old_checkout()
```

Real Example — GitHub Scientist GitHub uses “Scientist” library to run two code paths simultaneously (old + new), compare results, log differences — without affecting users. They validated their MySQL→MariaDB migration by running both in parallel.

22. The Complete Reliability Stack

Reliability Engineering Stack (bottom-up):

Infrastructure:

- └─ Multi-AZ deployment (hardware failure)
- └─ Multi-region (data center failure)
- └─ CDN (edge caching, DDoS)

Platform:

- └─ Kubernetes with HPA (autoscaling)
- └─ Circuit breakers (service failures)
- └─ Retries with exponential backoff + jitter
- └─ Graceful shutdown (SIGTERM → drain → exit)

Observability:

- └─ OpenTelemetry traces
- └─ Prometheus metrics + Grafana dashboards
- └─ Centralized logging (ELK/Loki)
- └─ Alerting (PagerDuty/OpsGenie)

Process:

- └─ SLO + Error Budget
- └─ On-call rotation
- └─ Incident response runbooks
- └─ Post-mortems (blameless)

PART 4 — System Design Interviews (Intermediate Level)

23. Design a Rate Limiter (Full Walkthrough)

This is one of the most commonly asked questions at Anthropic/OpenAI.

Requirements:

- Limit each user to 100 requests/minute
- Work across multiple API servers
- Minimal latency overhead (<5ms)

- Handle 1M requests/second

Architecture:

```
Client → [API Server N] → [Rate Limiter (Redis)] → [Backend]
                        ↑
                    Shared state for
                    distributed rate limiting
```

Implementation (Sliding Window Counter in Redis):

```
def is_allowed(user_id: str, limit: int = 100, window: int = 60) -> bool:
    now = time.time()
    window_start = now - window
    key = f"ratelimit:{user_id}"

    pipe = redis.pipeline()
    # Remove entries older than window
    pipe.zremrangebyscore(key, 0, window_start)
    # Count entries in window
    pipe.zcard(key)
    # Add current request
    pipe.zadd(key, {str(now): now})
    # Set expiry
    pipe.expire(key, window)

    results = pipe.execute()
    count = results[1]

    return count < limit
```

Headers to return:

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 47
X-RateLimit-Reset: 1234567890
Retry-After: 45 (on 429 response)
```

Multi-tier Rate Limiting:

```
Tier 1: IP-based (1000/min) ← DDoS protection at edge
Tier 2: API Key-based (100/min) ← Per customer
Tier 3: User-based (50/min) ← Per user within customer
Tier 4: Endpoint-based (/generate: 10/min, /embed: 100/min) ← Per feature
```

FDE Context: The Claude API uses multi-tier rate limiting. Enterprise customers negotiate higher limits. Rate limit data must be accurate across all API servers — requires centralized Redis, not in-memory per server.

24. Design a Notification System

Requirements: Send push, email, SMS, Slack notifications. 100M users. 1B notifications/day.

Architecture:

```
Trigger Service
  ↓ (event: "order shipped")
[Notification Service]
  ↓
[Kafka Topic: notifications]
  ↓           ↓           ↓
[Email Worker] [Push Worker] [SMS Worker]
  ↓           ↓           ↓
[SendGrid]    [FCM/APNs]    [Twilio]
```

Key decisions:

1. **Async via Kafka:** Decouple triggering from sending. If Twilio is slow, email still works.
2. **User preferences:** Check before sending: `user.notifications.email = true`
3. **De-duplication:** Idempotency key prevents duplicate sends on retry
4. **Priority queues:** Fraud alerts (high), marketing emails (low)
5. **Rate limiting per channel:** Twilio has its own rate limits
6. **Template service:** Centralized notification templates, localization

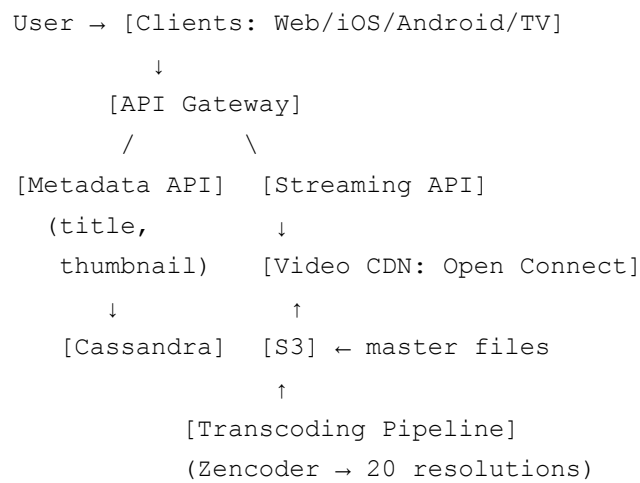
DLQ for failed notifications:

```
Email fails → retry 3x → DLQ
DLQ worker → inspect → manual retry or alert eng team
```

25. Design Netflix Video Streaming

Requirements: 220M subscribers. 15% of global internet traffic. 4K video. 99.99% availability.

Architecture (Simplified):



Key design decisions:

Adaptive Bitrate Streaming (ABR):

Video split into 4-second segments.
Each segment encoded at: 240p, 480p, 720p, 1080p, 4K

Player starts at 480p.
Measures: $\text{download_speed} / \text{segment_size} = \text{buffer_speed}$
If $\text{buffer_speed} > 1.5 \times \text{playback_speed}$ → upgrade to 720p
If $\text{buffer_speed} < 1.0 \times \text{playback_speed}$ → downgrade to 480p

Pre-positioning content: Netflix Open Connect appliances are in ISP buildings. Netflix sends popular content to the appliance *before* users request it (based on viewing predictions). When you click play, content is often <10ms away.

Multi-region availability:

Primary: us-east-1
Replica: us-west-2, eu-west-1, ap-northeast-1

If us-east-1 goes down:
DNS failover to us-west-2
Cassandra global tables already have data
CDN still serves cached content
Static content from S3 cross-region replication

Next: [Advanced Track →](#) — AI infrastructure, production-grade LLM systems, and the exact architecture questions asked at Anthropic and OpenAI.